

3.7 - gRPC e GraphQL

A ferramenta certa para cada camada

Dois protocolos, dois problemas

Comunicação interna versus clientes variados

- **gRPC usa Protocol Buffers sobre HTTP/2**
 - Mensagem binária compacta com contrato fortemente tipado
- **gRPC brilha na comunicação interna de serviços**
 - Alto volume e baixa latência entre back-ends
- **GraphQL deixa o cliente escolher os campos**
 - Uma chamada traz exatamente o que a tela precisa
- **GraphQL resolve over-fetching e under-fetching**
 - Fim das três chamadas para montar uma única tela

Onde cada um se encaixa

gRPC

- Comunicação serviço a serviço interna
- Alto throughput, latência mínima
- Contrato rígido definido em arquivo .proto
- Streaming bidirecional nativo
- Pouco amigável a navegador direto

GraphQL

- Clientes com necessidades de dados variadas
- App mobile e web como consumidores
- Schema flexível consultado pelo cliente
- Uma chamada agrega várias fontes
- Cache HTTP mais difícil de aplicar

Dois problemas, duas respostas

- **Rotas e telemetria trocam cinquenta mil mensagens por minuto**
 - App dos motoristas precisa de dados diferentes por tela

LUNALOG

A LunaLog tinha dois problemas de comunicação que pareciam o mesmo, mas não eram. O serviço de rotas e o de telemetria trocavam cinquenta mil mensagens por minuto, formato fixo, exigindo latência mínima, e ali o gRPC foi a resposta óbvia. Já o app dos motoristas precisava de dados diferentes em cada tela, e antes fazia três chamadas separadas para montar uma só. Com GraphQL, cada tela passou a pedir exatamente o que precisava em uma única chamada. A lição foi parar de procurar o protocolo certo e começar a procurar o protocolo certo para cada contexto.

Não é REST contra o resto

- REST na borda pública, fácil de cachear e documentar
- gRPC no miolo, GraphQL onde o cliente varia muito

A arquitetura madura mistura protocolos por camada, em vez de eleger um vencedor único. A pergunta certa é qual problema esta fronteira específica precisa resolver.



Com as APIs bem desenhadas, falta proteger o que está por trás delas. Imagine um cliente, ou apenas um bug de integração, disparando oitocentas chamadas por segundo no seu serviço. Sem defesa, esse único cliente derruba a experiência de todos os outros. Existe um conjunto de algoritmos feito exatamente para isso.

Rate limiting: proteger o serviço de abusos